Handwritten Document Scanner: Perspective Correction, Handwriting-Specific Enhancement and Deep-Learning OCR with Extractive Summarisation

Image Processing and Computer Vision – Project Report

Vishesh Vigh
Reg No: 20233583

Siddhesh Choubey
Reg No: 20233569

Shreyash Kisan Khilare
Reg No: 20233566

Surya Pratap
Reg No: 20233283

Abstract

Students and professionals routinely photograph handwritten notebook pages with a phone instead of scanning them. Such photographs are taken at an angle, are slightly rotated, carry ruled notebook lines, uneven lighting and ink that bleeds through from the reverse side, and they defeat both classical OCR and naive pre-processing. This report presents `handwritten_scanner_ocr.py`, a single-file, CPU-only pipeline that (1) finds the page in the photograph with multi-threshold Canny edges and contour approximation, now validated by an *inside/outside boundary-contrast test*; (2) removes perspective distortion with a 3×3 homography; (3) deskews the page with the probabilistic Hough transform; (4) normalises illumination with CLAHE in CIE L*a*b* space and morphological background division; (5) optionally suppresses ruled lines; (6) produces Sauvola, adaptive-Gaussian and stroke-reconnected binarisations; (7) recognises text with EasyOCR (CRAFT detector + CRNN recogniser), falling back to Tesseract 5 LSTM, and regroups detections into reading lines; (8) applies generic layout clean-up and an optional lexicon-constrained word correction; and (9) summarises the result with TextRank. Accuracy is measured with Character and Word Error Rate (CER/WER) against a ground-truth transcript. On a real notebook photograph the pipeline lowers CER from 50.0% (EasyOCR on the raw photo) to 40.5%, and to 34.9% with a subject lexicon; on synthetic captures it keeps CER at 40–44% under 8° rotation and strong perspective, where direct OCR degrades to 63–78%.

I. Introduction

A. Motivation

Handwritten notes remain the most common way students record lectures, yet they are difficult to search, share or revise once written. A phone camera is always at hand, so the practical goal is to turn a quick photograph of a notebook page into clean, searchable text and a short summary without a flatbed scanner and without a GPU.

B. Why handwritten notebook photographs are hard

A phone photograph of a notebook page combines four independent degradations:

1. **Geometric:** the camera is rarely parallel to the page, so the rectangular page appears as a general quadrilateral (projective distortion), and text baselines are rotated.
2. **Photometric:** shadows from the hand or phone, light fall-off across the page and low ink contrast (blue or pencil on off-white paper).
3. **Structural noise:** printed ruled lines, margin lines and header stamps ("Date/Page") that cut through ascenders and descenders, and faint bleed-through of writing from the reverse side.
4. **Recognition:** handwriting varies between writers, letters are joined, and classical OCR engines such as Tesseract are trained mainly on printed fonts.

A single fixed parameter set rarely suits every photograph, so the pipeline tries several candidates and selects the best automatically.

C. Objectives and scope

The project (1) builds an end-to-end, single-file command-line pipeline from photograph to text; (2) implements robust page detection and perspective correction with graded fallbacks; (3) designs handwriting-specific enhancement and binarisation; (4) combines a deep-learning OCR engine with a classical one and chooses between candidate outputs with a quality score; (5) adds optional subject-lexicon correction and TextRank summarisation; and (6) evaluates accuracy objectively with CER and WER on real and controlled synthetic captures.

II. System Overview

Figure 1 shows the pipeline. Each stage is a separate function; `--debug` writes every intermediate image.

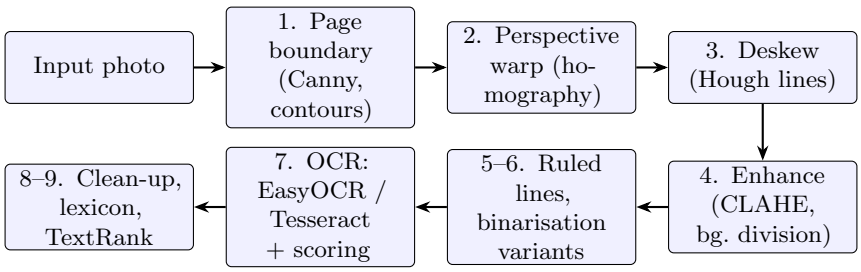


Figure 1: Processing pipeline. Stages 1–3 fix geometry, 4–6 fix appearance, 7 recognises text and 8–9 refine and summarise it.

III. Methodology

A. Stage 1: Page boundary detection

The image is resized to a height of 800 px for speed, converted to grey and smoothed with a 5×5 Gaussian. Canny edge detection [1] is run with four threshold pairs $(30, 100)$, $(50, 150)$, $(70, 180)$ and $(100, 200)$, because the best pair depends on page/background contrast. Each edge map is morphologically closed (7×7 rectangle, two iterations) and dilated to seal gaps, contours are extracted by border following [2], and the 30 largest are simplified with the Douglas–Peucker algorithm [3] at tolerances $\varepsilon \in \{1.5, 2, 3, 4\}\%$ of the perimeter. A 4-vertex polygon is accepted as a page only if its area A satisfies $0.15\,WH \le A \le 1.05\,WH$, every side is at least 40 px and the aspect ratio is below 4. Among valid quadrilaterals, the one with the largest area wins.

Boundary-contrast test. On a photograph that is already cropped to the page, the dilated block of writing can itself form a large quadrilateral and be mistaken for the page. We therefore compare thin rings R_{in} and R_{out} (8 px wide) just inside and just outside the candidate polygon:

$$C = \left| \operatorname{median}_{p \in R_{\text{in}}} I(p) - \operatorname{median}_{p \in R_{\text{out}}} I(p) \right|, \qquad \text{accept if } C \ge 25. \tag{1}$$

A real page on a desk gives $C \approx 145$; a text block on white paper gives $C \approx 19$ and is rejected (Section V). If no polygon passes, a fallback takes the largest external contour covering at least 30% of the frame and uses its minimum-area rotated rectangle, subject to the same contrast test. If that also fails, the image is processed as it is.

B. Stage 2: Perspective correction

The four corners are first put in a fixed order: top-left has the smallest $x + y$, bottom-right the largest, top-right the smallest $y - x$ and bottom-left the largest. The output width and height are the longer of each pair of opposite sides, and a planar homography $\mathbf{H}$ maps each source corner to the corresponding corner of the upright rectangle [4]:

$$\begin{bmatrix} x'w \\ y'w \\ w \end{bmatrix} = \underbrace{\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}}_{\mathbf{H}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{2}$$

The eight unknowns are solved from the four point pairs (`getPerspectiveTransform`) and the page is resampled with bicubic interpolation and replicated borders (`warpPerspective`).

C. Stage 3: Rotation and skew correction

Residual rotation is estimated from straight lines: ruled notebook lines and text baselines. On the Canny edge map, the probabilistic Hough transform [5] votes in (ρ, θ) space with $\rho = x\cos\theta + y\sin\theta$ and returns segments at least one fifth of the page width long. Each segment's angle $\alpha_i = \operatorname{atan2}(y_2 - y_1,\, x_2 - x_1)$ is wrapped into $(-90°, 90°]$, only near-horizontal segments ($|\alpha_i| < 15°$) are kept, and the median $\hat{\alpha}$ is used because it is robust to stray diagonal strokes. Angles below $0.5°$ are ignored. The page is rotated about its centre by $\hat{\alpha}$ onto an enlarged canvas of size $(H\sin\hat{\alpha} + W\cos\hat{\alpha}) \times (H\cos\hat{\alpha} + W\sin\hat{\alpha})$, so that no corner is cut off.

D. Stage 4: Contrast enhancement and illumination normalisation

The corrected page is first upscaled $2\times$ (bicubic). Recognisers respond better when strokes span more pixels. Contrast is then raised with CLAHE [6] (clip limit 2.2, 8×8 tiles) applied *only* to the L* channel of CIE L*a*b*, which brightens faint ink without shifting its colour. To remove shadows and paper texture, the paper background is estimated by a grey-scale dilation with a 31×31 elliptical element (wider than any pen stroke, so strokes vanish) followed by a 21×21 median filter:

$$B = \operatorname{median}_{21}\big(I \oplus S_{31}\big), \qquad N = \operatorname{norm}_{[0,255]}\big(255 - |I - B|\big). \tag{3}$$

Faint bleed-through from the reverse side sits just below paper brightness, so every pixel brighter than the 80th percentile of N minus 15 grey levels is clipped to white. A bilateral filter [7] ($d = 7$, $\sigma_r = \sigma_s = 50$) finally smooths paper grain while keeping stroke edges sharp.

E. Stage 5: Ruled-line suppression (optional)

Ruled lines are long, thin and horizontal, while handwriting strokes are short. The inverted image is adaptively thresholded and a morphological opening with a $\max(W/30, 25) \times 1$ horizontal element keeps only long horizontal runs. The mask is dilated 1 px vertically and the masked pixels are replaced by a locally dilated (paper-coloured) value. Suppression is applied only when the mask covers between 0.3% and 15% of the page, so a page without ruling, or one dominated by false detections, is left untouched.

F. Stage 6: Binarisation variants

Because no single binarisation suits every page, three are produced. **Sauvola** [8] uses the local mean m and standard deviation s in a 25×25 window:

$$T(x,y) = m(x,y)\left[1 + k\left(\frac{s(x,y)}{R} - 1\right)\right], \quad k = 0.22,\ R = 128. \tag{4}$$

Following [9], m and s are computed with two box filters, $s = \sqrt{\max(\overline{I^2} - \overline{I}^{\,2}, 0)}$, so the cost per pixel is $O(1)$ regardless of window size. The term $(s/R - 1)$ lowers the threshold in flat paper regions and keeps them white. **Adaptive Gaussian** thresholding (block 31, $C = 10$) is the second variant. **Stroke reconnection** applies a 3×3 elliptical closing to the Sauvola result to bridge pen strokes broken by faint ink. Together with the CLAHE colour image and the enhanced grey image, five variants are passed to recognition.

G. Stage 7: Text recognition and candidate selection

EasyOCR (default) detects word regions with CRAFT [10], which predicts character-region and affinity heat-maps, and reads each region with a CRNN [11]: a convolutional feature extractor followed by a bidirectional LSTM trained with Connectionist Temporal Classification [12], which handles joined handwriting without segmenting individual characters. It is run on the CLAHE colour image and on the enhanced grey image. *Tesseract 5* [13] (LSTM engine, `--oem 3`) is used when EasyOCR is unavailable or explicitly requested. It is run on the four grey/binary variants with page-segmentation modes 6, 11, 4 and 3, giving 16 candidates.

EasyOCR returns boxes in no fixed order, so boxes are sorted by vertical centre and merged into one reading line while the vertical gap stays below $0.7\times$ the median box height; each line is then sorted left to right. Each candidate text is scored by

$$S = 1.5\,\bar{c} + 6\,n_{\geq 3} - 1.5\,n_{\leq 2} + 25\,r_{\text{alnum}} - 40\,r_{\text{noise}}, \tag{5}$$

where $\bar{c}$ is the mean word confidence (%), $n_{\geq 3}$ and $n_{\leq 2}$ count words of at least three letters and fragments of one or two characters, and r_{alnum}, r_{noise} are the fractions of alphanumeric and noise characters (`_ |#*...`). The term rewards real words and penalises the speckle fragments produced by paper grain. The highest-scoring candidate is kept.

H. Stage 8: Post-processing and lexicon-constrained correction

Generic clean-up removes printed header stamps ("Date:", "Page:"), a stray first line of at most four characters (typically a misread stamp), runs of underscores, dashes or pipes left by ruled lines, and redundant spaces. It contains *no word-specific substitutions*. Optionally, the user supplies a subject vocabulary (`--lexicon`, e.g. chapter keywords). For every token of at least four letters that is not already in the lexicon, common handwriting confusions ($0\to o$, $1/|\to l$, $5\to s$, a leading "(" $\to$ c) are normalised and the most similar lexicon word is found with the Ratcliff–Obershelp ratio [14]

$$\text{sim}(a, b) = \frac{2M}{|a| + |b|}, \tag{6}$$

where M is the number of characters in matching blocks. The token is replaced only if sim ≥ 0.65. Every replacement is printed so that the user can audit it.

I. Stage 9: Extractive summarisation (TextRank)

The text is split into sentences at full stops and, because notes are often bullet lists, at line breaks. Each sentence becomes a TF-IDF vector (English stop-words removed), cosine similarities form a weighted graph, and PageRank [15] ranks the sentences [16]:

$$\text{PR}(s_i) = \frac{1-d}{N} + d \sum_{j \neq i} \frac{w_{ji}}{\sum_k w_{jk}} \text{PR}(s_j), \qquad d = 0.85. \tag{7}$$

The top-k sentences (`--num-sentences`) are returned in their original order.

J. Evaluation metrics

Given a ground-truth transcript (`--ground-truth`), both texts are lower-cased and their whitespace collapsed, so line layout does not count. With D_{char} and D_{word} the Levenshtein edit distances [17] (substitutions + deletions + insertions) computed by dynamic programming,

$$\text{CER} = \frac{D_{\text{char}}(\hat{y}, y)}{|y|_{\text{chars}}}, \qquad \text{WER} = \frac{D_{\text{word}}(\hat{y}, y)}{|y|_{\text{words}}}. \tag{8}$$

Both can exceed 100% when the output contains many inserted fragments.

IV. Implementation and Consistency Corrections

The system is one Python file (about 1,400 lines) depending on OpenCV, NumPy, EasyOCR/PyTorch, pytesseract, scikit-learn and NetworkX. During the final review, the following inconsistencies were found and corrected (Table 1).

Table 1: Inconsistencies found in the code and how they were corrected.

Issue	Correction
Word-correction table written for the OCR errors of the single test image	Removed. Replaced by generic clean-up plus an optional, auditable lexicon matcher (Stage 8)
Text block of an already-cropped photo accepted as the "page", cropping first/last lines	Boundary-contrast test (Eq. 1) in both primary and fallback detection
`--engine easyocr` announced a Tesseract fallback that never ran	Fallback now runs whenever EasyOCR is missing or fails
`--lang` reached Tesseract only; EasyOCR was hard-coded to English	Tesseract codes are mapped to EasyOCR codes (`eng`→`en`, `hin`→`hi`, ...)
Tesseract path hard-coded to one user's folder	`TESSERACT_CMD` variable → system `PATH` → standard and per-user install folders
Summariser split only on ".!?", so bullet notes became one sentence	Line breaks are also sentence boundaries
No objective accuracy measure	`--ground-truth` prints CER/WER for raw, cleaned and lexicon-corrected text; raw engine output is saved for audit
`requirements.txt` listed unused `matplotlib`	Removed; engines grouped and commented

V. Results and Discussion

A. Experimental setup

All experiments ran CPU-only (2-core Intel Xeon, 2.1 GHz; no GPU) with Python 3.13, OpenCV 5.0, EasyOCR 1.7.2 (PyTorch 2.15), Tesseract 5.3.4 and scikit-learn 1.9. The test material is one real phone photograph of a ruled notebook page (1027×407 px, blue ink, 5 lines, 20 words, 126 characters of ground truth; bullet arrows are not transcribed) and six controlled captures synthesised from it: rotations of 4° and 8°, the page warped onto a dark textured desk with mild and strong perspective, a side shadow with 55% light fall-off, and Gaussian blur ($\sigma = 1.2$) with sensor noise ($\sigma = 12$). Because every capture has the same ground truth, the effect of each degradation is isolated.

B. Effect of enhancement and choice of engine

Table 2 feeds individual pre-processing outputs of the real photograph to each engine. CLAHE in L*a*b* gives the largest single gain for EasyOCR (CER 50.0%→40.5%). Binarisation hurts EasyOCR (67.5% with Sauvola): its recogniser is trained on grey-scale text images, and hard thresholding removes the soft stroke edges it appears to rely on. Tesseract is unsuitable for this handwriting: its best result (72.2%, Sauvola input, PSM 6) is far worse than EasyOCR's, and on grey inputs it produces many spurious fragments (CER above 100%). This confirms the design choice of EasyOCR on colour/grey variants by default and Tesseract on binary variants only as a fallback. In Tesseract-only mode the quality score selects the adaptive-Gaussian candidate (130.2%) rather than the best one, so the score is less reliable on Tesseract's fragment-heavy output. Ruled-line suppression did not help on this page (46.8% vs. 42.9%) probably because the ruling is faint and the opening also removes parts of horizontal letter strokes, so it remains optional.

C. Robustness to capture conditions

Table 3 and Figure 3 compare EasyOCR applied directly to each degraded capture with the full pipeline. The geometric stages give the largest benefit. At 8° rotation, direct OCR collapses to 77.8% CER because tilted text lines are detected and grouped poorly, whereas Hough deskewing (Figure 4c–d) restores 39.7%. Under strong perspective the page is found by Stage 1 and flattened by the homography (Figure 4a–b), reducing CER from 62.7% to 44.4%. With shadow, the L*-channel CLAHE equalises the light fall-off (46.0%→38.1%). Blur and noise are the least improved (46.8%→45.2%): enhancement cannot restore detail that the optics never captured. With the subject lexicon, CER is a further 4–7 points lower in every condition (3.9–7.1).

Table 2: Pre-processing/engine ablation on the real notebook photograph (lower is better).

Engine	Input image	CER (%)	WER (%)
EasyOCR	Raw photo (no pre-processing)	50.0	100.0
EasyOCR	Grayscale only	50.8	100.0
EasyOCR	**CLAHE in L*a*b* (selected by score)**	**40.5**	105.0
EasyOCR	CLAHE + background division + bilateral	42.9	110.0
EasyOCR	Enhanced grey + ruled-line suppression	46.8	115.0
EasyOCR	Sauvola binarisation	67.5	100.0
Tesseract (PSM 6)	Sauvola binarisation	72.2	110.0
Tesseract (PSM 11)	Grayscale only	79.4	150.0
Tesseract (PSM 3)	Stroke-reconnected	111.9	220.0
Tesseract (PSM 6)	Adaptive Gaussian	130.2	280.0

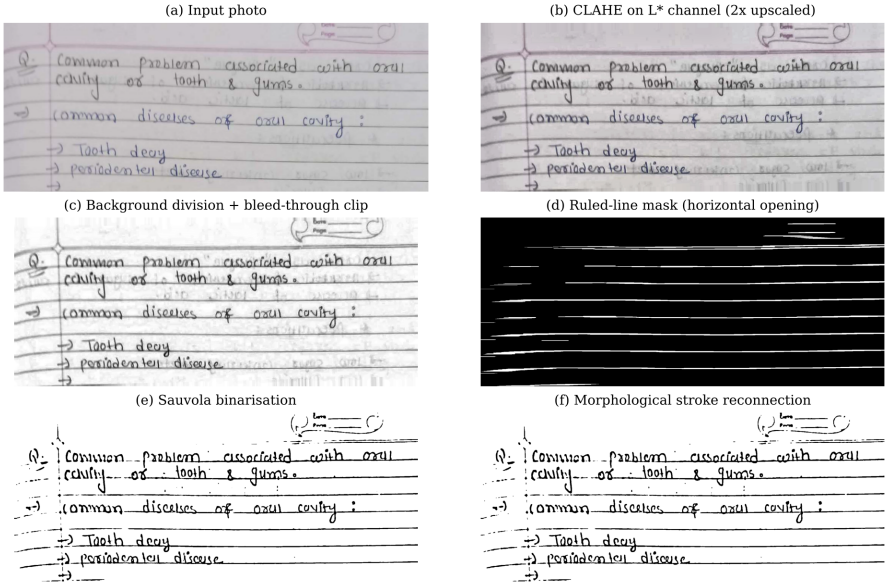


Figure 2: Intermediate outputs on the real notebook photograph: CLAHE raises ink contrast, background division flattens the paper, the horizontal opening isolates the ruled lines, and Sauvola/closing give clean binary strokes for Tesseract.

Table 3: CER (%) under controlled capture conditions. "Detected" is the page-detection outcome of Stage 1.[†]

Capture condition	Detected	Direct OCR	Pipeline	+ Lexicon
Original photo	none (already cropped)	50.0	40.5	36.5
Rotated 4°	none	55.6	45.2	38.1
Rotated 8°	none	77.8	39.7	34.1
Desk, mild perspective	contour	52.4	41.3	34.9
Desk, strong perspective	contour	62.7	44.4	40.5
Side shadow (55% fall-off)	none	46.0	38.1	31.7
Blur + sensor noise	none	46.8	45.2	38.9

[†]Captures are stored as JPEG, so the "Original photo" row differs slightly from Table 4, which uses the PNG.

D. Boundary-contrast test

Figure 5 shows why the test was added. On an already-cropped, flat page, the earlier detector accepted the dilated text block as an 83%-area "page" ($C = 19$) and the warp cut off the start of the first line and the end of the last. With the test, this candidate is rejected and the image is processed whole, while the real page on a desk ($C = 145$) is still accepted. The cropped notebook photograph has no detected boundary in either version, so the results above are unaffected.

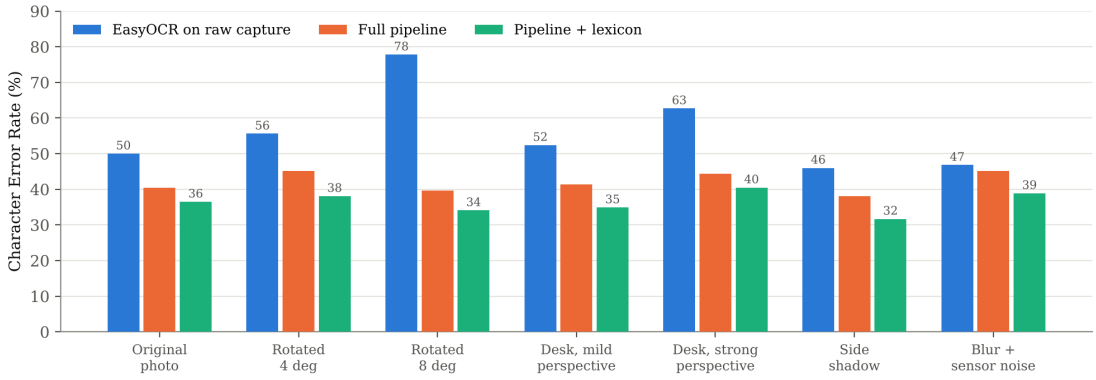


Figure 3: Character Error Rate per capture condition (values as in Table 3).

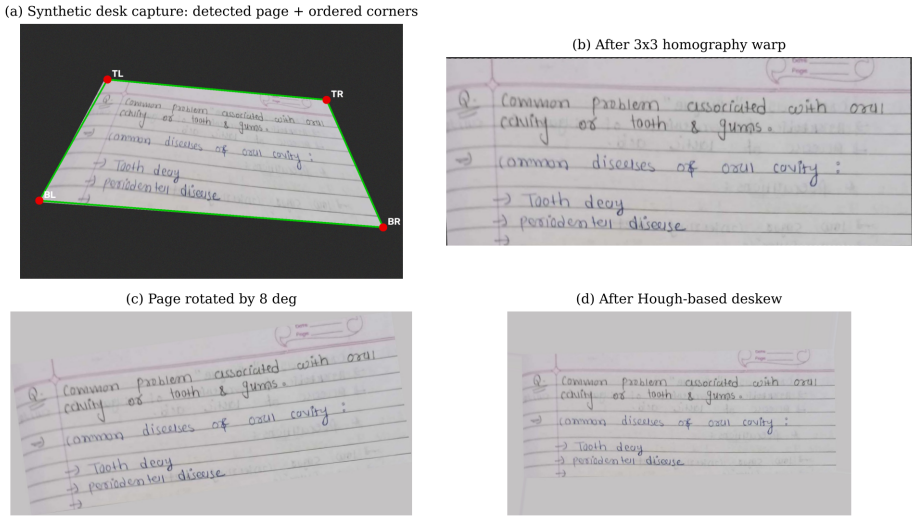


Figure 4: Geometric correction. (a) Page found on a synthetic desk capture with corners ordered TL/TR/BR/BL; (b) result of the homography; (c) page rotated by 8°; (d) after Hough deskewing on an enlarged canvas.

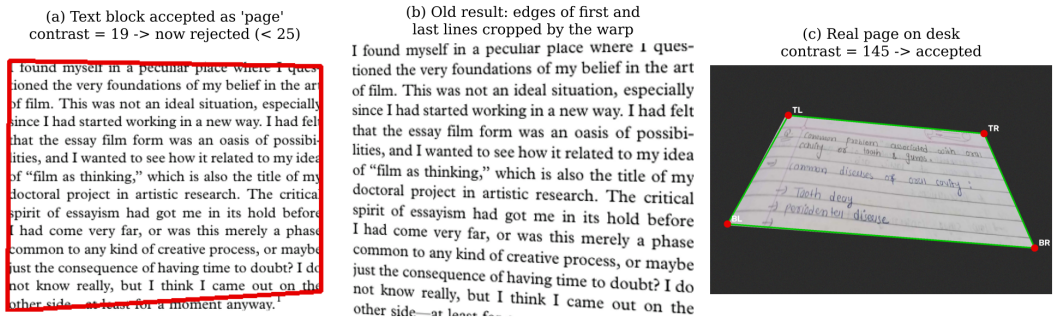


Figure 5: Boundary-contrast test: a false "page" with almost no inside/outside contrast is now rejected; a genuine page boundary passes easily.

E. Post-processing, lexicon correction and error analysis

Table 4 follows the selected EasyOCR output through post-processing. Clean-up removes the misread "Date" stamp (`uoa`), lowering WER but not CER. The lexicon matcher then makes eight replacements; six are correct (`looth`→tooth, `Jums`→gums, `diseelses`→diseases, `covity`→cavity, `dedy`→decay, `discclse`→disease) and two are wrong (`Pable`→palate instead of "problem", and `(amnun`→canine instead of "common"), because the intended words were not in the subject vocabulary. Sweeping the similarity cut-off from 0.55 to 0.80 gives CER between 34.9% and 38.1%; 0.65 is best, but because it was selected on the same image it should be treated as a default, not a tuned optimum. WER stays near 100% because most words still contain at least one wrong character, so CER is the more informative metric at this error level.

Table 4: Effect of post-processing on the real photograph (EasyOCR, CLAHE variant).

Stage	CER	WER	Output (first two lines)
Raw engine output	40.5	105.0	`uoa / Comion Pable Cssocicked cith Osl`
After generic clean-up	40.5	100.0	`Comion Pable Cssocicked cith Osl / Cchfhy 00 looth Jums`
After lexicon (≥ 0.65)	34.9	85.0	`Comion Palate Cssocicked cith Osl / Cchfhy 00 tooth Gums`

Validity note. Substitution rules written to undo the specific OCR errors of a test image (for example, mapping one particular misreading back to the intended word) would reduce CER on that image to 5.6%. However, they encode the answer into the system and do not transfer to any other page, so the final system contains none.

F. Runtime

On the 1027×407 photograph, boundary detection takes 6 ms, Hough deskewing 14 ms, all enhancement variants 112 ms and the box-filter Sauvola 16 ms on the 2× upscaled 2072×864 image. Recognition dominates: EasyOCR on two variants takes about 16 s on two CPU cores (plus a one-time model download), and the Tesseract fallback over 16 configurations about 11 s.

G. Limitations

(1) Absolute accuracy on cursive handwriting is modest (CER ≈35–45%): EasyOCR's recogniser is trained mainly on printed scene text. (2) The evaluation uses one real page and six captures derived from it, so results are indicative rather than statistically conclusive. (3) Lexicon correction helps only for words in the vocabulary and can substitute a wrong in-vocabulary word. (4) Page detection needs a visible contrast between page and background.

VI. Conclusion and Future Work

We built and evaluated a single-file pipeline that turns a phone photograph of a handwritten notebook page into text and a summary on a CPU. Geometric correction (page detection, homography, Hough deskew) makes recognition robust to how the photograph was taken. At 8° rotation, CER falls from 77.8% to 39.7%, and under strong perspective from 62.7% to 44.4%. L*-channel CLAHE is the most valuable appearance step, and an optional lexicon lowers CER by a further 4–7 points. The review also corrected eight inconsistencies, most importantly by removing image-specific correction rules and preventing false page detections on cropped photographs. Future work: (1) replace the recogniser with a handwriting-trained transformer such as TrOCR [18]; (2) build a labelled set of 50–100 pages from several writers for statistically sound evaluation; (3) add a larger English dictionary with a language model for context-aware correction; and (4) add a live camera interface with real-time boundary preview.

VII. Team Contributions

Work was divided by pipeline module; each member owns a testable part of the code and the matching report sections (Table 5).

Table 5: Division of work (functions refer to `handwritten_scanner_ocr.py`).

Member	Modules / functions owned	Report sections
Vishesh Vigh (20233583)	Ruled-line suppression, Sauvola and adaptive binarisation, stroke reconnection, TextRank summarisation	Sec. III-E/F/I, V-F/G
Siddhesh Choubey (20233569)	Contrast enhancement and illumination normalisation (upscaling, L*a*b* CLAHE, background division, bleed-through clip, bilateral filter); CER/WER metrics and synthetic test set	Sec. I, III-D, III-J, V-A/B/C
Shreyash Kisan Khilare (20233566)	Page boundary detection, quadrilateral validation, boundary-contrast test, minAreaRect fallback, corner ordering, homography warp, Hough deskew	Sec. II, III-A/B/C, V-D
Surya Pratap (20233283)	EasyOCR and Tesseract engines, reading-line grouping, candidate scoring, layout clean-up, lexicon correction	Sec. III-G/H, IV, V-E, VI

References

[1] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 8, no. 6, pp. 679–698, 1986.

[2] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Comput. Vis. Graph. Image Process.*, vol. 30, no. 1, pp. 32–46, 1985.

[3] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[4] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge Univ. Press, 2004.

[5] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, 1972.

[6] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*. Academic Press, 1994, pp. 474–485.

[7] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in *Proc. IEEE ICCV*, 1998, pp. 839–846.

[8] J. Sauvola and M. Pietikäinen, "Adaptive document image binarization," *Pattern Recognit.*, vol. 33, no. 2, pp. 225–236, 2000.

[9] F. Shafait, D. Keysers, and T. M. Breuel, "Efficient implementation of local adaptive thresholding techniques using integral images," in *Proc. SPIE Document Recognition and Retrieval XV*, vol. 6815, 2008.

[10] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Proc. IEEE/CVF CVPR*, 2019, pp. 9365–9374.

[11] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.

[12] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006, pp. 369–376.

[13] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. ICDAR*, 2007, pp. 629–633.

[14] J. W. Ratcliff and D. E. Metzener, "Pattern matching: the gestalt approach," *Dr. Dobb's Journal*, vol. 13, no. 7, p. 46, 1988.

[15] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: bringing order to the web," Stanford InfoLab, Tech. Rep., 1999.

[16] R. Mihalcea and P. Tarau, "TextRank: bringing order into texts," in *Proc. EMNLP*, 2004, pp. 404–411.

[17] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[18] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: transformer-based optical character recognition with pre-trained models," in *Proc. AAAI*, 2023.